



**ITM SKILLS
UNIVERSITY**

School of Future Tech

Case Study Report

on

CURRENCY & UNIT CONVERTER

by

Aditya khare

Roll No: 150096725061

[Github: https://github.com/adiii-0710/currency-unit-converter-java](https://github.com/adiii-0710/currency-unit-converter-java)

CURRENCY & UNIT CONVERTER

1. Introduction

The Currency & Unit Converter is a Java-based console application that allows users to convert currency and measurement values through a simple menu-driven interface. The program performs INR to USD, Kilometers to Miles, and Kilograms to Pounds conversions.

The application runs locally in the terminal and focuses on core Java programming concepts such as switch statements, methods, wrapper classes, loops, user input, and exception handling. Input is read using Scanner and converted safely using Integer.parseInt() and Double.parseDouble().

2. Objective of the Project

The objective of this project was to build a simple and reliable Java converter that demonstrates operators, methods, switch-based menus, wrapper classes, and safe input validation.

- Provide a menu-driven interface for multiple conversions.
- Convert INR to USD using a fixed demonstration exchange rate.
- Convert kilometers to miles.
- Convert kilograms to pounds.
- Use a reusable convert() method for conversion logic.
- Handle invalid menu and numeric inputs without crashing.
- Keep the program running until the user selects Exit.

3. Technologies Used

Area	Details
Programming Language	Java
Input Handling	java.util.Scanner
Core Concepts	Switch, Methods, Loops, Wrapper Classes, Exception Handling

The project does not require a database, backend server, external API, or additional framework.

4. Program Organization

The application is organized around a main menu and a reusable conversion method. The main() method controls user interaction, validation, and menu flow, while convert() contains the actual conversion formulas.

Component	Responsibility
CurrencyUnitConverter.java	Contains the complete converter application.
main()	Displays the menu, reads input, validates values, and controls program flow.
convert()	Performs INR-USD, KM-Miles, and KG-Pounds conversions.
Scanner	Reads user input from the terminal.

5. Conversion Logic

The convert() method accepts a numeric value and a character representing the conversion type. A switch statement selects the appropriate formula.

Type	Conversion	Formula
I	INR → USD	value / 83
K	KM → Miles	value × 0.621

G	KG → Pounds	value × 2.205
---	-------------	---------------

The INR to USD rate is fixed at 83 in this assignment and is intended for demonstration. The program does not fetch live exchange rates.

6. Menu and Switch-Based Control

The program displays four options: INR-USD, KM-Miles, KG-Pounds, and Exit. The selected menu option is processed using a switch statement. A while(true) loop allows repeated conversions until option 4 is selected.

- Choice 1 calls convert(value, 'I') for INR to USD.
- Choice 2 calls convert(value, 'K') for kilometers to miles.
- Choice 3 calls convert(value, 'G') for kilograms to pounds.
- Choice 4 terminates the loop and closes the Scanner.

7. Input Validation and Exception Handling

User input is initially read as text using Scanner.nextLine(). Integer.parseInt() is used to convert the menu choice, while Double.parseDouble() converts the entered value.

Both conversions are protected using try-catch blocks. If the user enters invalid data, NumberFormatException is caught and an appropriate message is displayed instead of allowing the program to terminate unexpectedly.

- Invalid menu input such as letters is rejected.
- Menu values outside the range 1-4 are rejected.
- Non-numeric conversion values are rejected.
- The user can continue using the converter after invalid input.

8. Concepts Demonstrated

Concept	Application in Project
Operators	Arithmetic operators are used in conversion formulas.
Switch	Controls conversion type and menu selection.
Methods	convert() provides reusable conversion logic.
Wrapper Classes	Integer.parseInt() and Double.parseDouble() parse input.
Exception Handling	NumberFormatException is handled using try-catch.
Looping	while(true) keeps the application active until Exit.

9. Sample Console Output

The following examples represent the expected terminal interaction of the application.

```
===== Currency & Unit Converter =====
1. INR-USD
2. KM-Miles
3. KG-Pounds
4. Exit
Choice: 1
Enter value: 830
830.00 INR = 10.00 USD
```

```
===== Currency & Unit Converter =====
1. INR-USD
2. KM-Miles
```

```
3. KG-Pounds
4. Exit
Choice: 2
Enter value: 10
10.00 km = 6.21 miles
```

```
===== Currency & Unit Converter =====
1. INR-USD
2. KM-Miles
3. KG-Pounds
4. Exit
Choice: 3
Enter value: 10
10.00 kg = 22.05 pounds
```

10. Results

The completed Java converter successfully accepts user input, performs the selected conversion, formats the result to two decimal places, and continues to provide conversions until the user exits.

- The menu operates through switch-based control.
- INR, kilometer, and kilogram conversions produce formatted numerical results.
- Invalid menu and numeric input is handled using `NumberFormatException`.
- The reusable `convert()` method keeps conversion formulas separate from menu interaction.
- The `Scanner` resource is closed when the application exits.

11. Conclusion

This project demonstrates the implementation of a practical console-based utility using core Java programming concepts. The Currency & Unit Converter combines arithmetic operators, switch statements, methods, wrapper classes, loops, and exception handling in one application.

Through this project, I learned how to design a menu-driven Java program, separate reusable logic into methods, safely parse user input, handle invalid data, and present calculated results in a clear format.